Proyecto 3

Text-to-SQL: fine-tuning de un LLM con LoRA

Universidad del Valle
Deep Learning 2026
Kevin Recinos

Asignación	lunes 12 de octubre de 2026
Entrega	domingo 25 de octubre de 2026, 23:59
Medio	repositorio privado en GitHub; en Canvas, el repositorio comprimido y el PDF
Presentación oral	no habrá
Modalidad	parejas
Ponderación	8 puntos sobre la nota final del curso

1. Objetivo

Hacer fine-tuning con LoRA de un LLM pequeño para que, dado el esquema de una base de datos y una pregunta en español, genere una consulta MySQL correcta.

Después de la entrega, el docente ejecutará el modelo de cada pareja sobre **bases de datos y preguntas que ustedes no conocen**. Ese resultado es parte de la nota.

2. Condiciones fijas

Modelo base. Uno de estos tres, de Hugging Face:

- `Qwen/Qwen2.5-Coder-1.5B-Instruct` (recomendado)
- `Qwen/Qwen2.5-Coder-0.5B-Instruct`
- `Qwen/Qwen3-1.7B` (con el modo de razonamiento desactivado)

Se entrega el modelo base más un adapter LoRA o QLoRA entrenado por la pareja. No se permite otro modelo ni el uso de APIs externas al generar la consulta.

Motor. MySQL **8.4.10**, con su configuración por defecto (incluye `ONLY_FULL_GROUP_BY`). Pueden instalarlo en su computadora o en la nube (Colab, Kaggle u otro). El binario para Linux está en:

`https://downloads.mysql.com/archives/get/p/23/file/mysql-8.4.10-linux-glibc2.17-x86_64-minimal.tar.xz`

También pueden usar la imagen de Docker `mysql:8.4.10`.

Entorno de evaluación. El docente ejecutará las entregas en Google Colab con GPU T4, instalando el `requirements.txt` de cada pareja. Desarrollen donde prefieran, pero verifiquen que su entrega corre en ese entorno.

3. Contrato de la entrega

El archivo `solucion.py` carga el modelo indicado en `config.json` y define esta función:

```
def generar_sql(esquema: str, pregunta: str, ejecutar=None) -> str:
    ...
```

- `esquema`: sentencias `CREATE TABLE` de la base, tal como las devuelve `SHOW CREATE TABLE`, incluidos los `COMMENT`.
- `pregunta`: texto en español.
- `ejecutar(sql)`: función opcional que entrega el evaluador. Ejecuta una consulta en la base, con un usuario de solo lectura, y devuelve un diccionario con `ok`, `columnas`, `filas` (máximo 10) y `error`. Permite hasta 3 llamadas por pregunta.
- Valor de retorno: **una sola** consulta `SELECT` o `WITH ... SELECT`.

Límites:

- 30 segundos por pregunta, como máximo.
- 5 minutos para cargar el modelo, como máximo.
- Generación determinista (`do_sample=False` o semilla fija).

`config.json` debe tener esta forma:

```
{
  "modelo_base": "Qwen/Qwen2.5-Coder-1.5B-Instruct",
```

```
  "adapter": "adapter/",
  "usar_adapter": true,
  "semilla": 2026
}
```

Con `"usar_adapter": false`, `solucion.py` debe usar el modelo base con el mismo prompt. El docente evaluará ambas versiones para medir el aporte del fine-tuning.

4. Evaluación oculta

- Una o más bases de 5 a 10 tablas. Los nombres pueden estar en español o en inglés, y pueden ser abreviados.
- Entre 30 y 40 preguntas en tres niveles:
  - **fácil**, con peso 1: una tabla;
  - **media**, con peso 2: `JOIN`, `GROUP BY`, `HAVING`;
  - **difícil**, con peso 3: subconsultas, `WITH`, funciones de ventana, fechas.
- Cada pregunta indica qué columnas devolver.
- **Criterio de acierto: exactitud por ejecución.** Se compara el resultado de su consulta con el de la consulta de referencia, no el texto.
  - El orden de las filas solo cuenta si la pregunta lo pide.
  - Los números se redondean a 2 decimales.
  - Una columna adicional no penaliza.
- Métrica: exactitud ponderada (EX), es decir, la suma de los pesos de las preguntas acertadas dividida entre la suma de todos los pesos.

Si la entrega no corre por un error de la pareja, se concede una sola corrección en 24 horas, con -10 puntos.

5. Qué deben hacer

1. **Dataset de entrenamiento** (al menos 2,000 pares pregunta–SQL). Pueden usar datasets públicos (Spider, BIRD), convertidos a MySQL y con las preguntas traducidas al español, y pares generados con un LLM sobre al menos 10 esquemas propios. Solo se aceptan pares cuya consulta se ejecute sin error en MySQL 8.4.10. Reporten cuántos pares descartaron y por qué.
2. **Validation set propio.** Al menos 2 esquemas que no aparezcan en el entrenamiento, con 40 preguntas escritas por ustedes en los tres niveles. La partición se hace por esquema, no por pregunta.
3. **Baseline:** el modelo sin adapter, evaluado en su validación.
4. **Dos configuraciones de LoRA** que difieran en un solo factor. Para cada una, escriban la hipótesis antes de entrenar y reporten los parámetros entrenables, el tiempo de entrenamiento y las curvas de loss.
5. **Un experimento de inferencia.** Por ejemplo: corregir la consulta con `ejecutar`, cambiar el formato del esquema o agregar ejemplos al prompt (few-shot).
6. **Análisis de errores por categoría:** sintaxis, columna o tabla inexistente, `JOIN`, agregación, filtro, o resultado distinto.
7. **Predicción justificada** de su EX en la evaluación oculta.

6. Colaboración

- Repositorio privado en GitHub, compartido con el docente desde el inicio. Ambos integrantes hacen *commits* desde su propia cuenta durante las dos semanas.
- `historial_git.txt`, generado con `git log --all --date=iso --stat`.
- `BITACORA.md`: al menos 8 entradas fechadas, cada una con su autor.
- `CONTRIBUCIONES.md`: tabla con tarea, responsable y evidencia. Cada integrante es responsable de al menos una de estas áreas: datos, entrenamiento, o validación y errores.
- Coevaluación individual en Canvas después de la entrega.

Si la evidencia muestra un aporte desigual, la nota individual puede ajustarse.

7. Entregables

```
solucion.py, config.json, requirements.txt, adapter/
datos/            entrenamiento, validación propia, descartes
notebooks/        datos, entrenamiento, evaluación
resultados/       validacion_propia.csv
```

```
presentacion.pdf  máximo 12 diapositivas
README.md         ejecución, licencias, uso de IA
BITACORA.md, CONTRIBUCIONES.md, historial_git.txt
```

8. Evaluación

Componente	Puntos
Evaluación oculta: EX con adapter (30) y mejora sobre la versión sin adapter (10)	40
Dataset, baseline y configuraciones de LoRA	25
Validación propia, experimento de inferencia y análisis de errores	20
Colaboración evidenciada	7
Presentación y predicción justificada	8
Total	**100**

Rendimiento del modelo (30 puntos): Si el modelo con adapter alcanza un EX de 0.70 o más, obtiene los 30 puntos. Si es menor, la nota será proporcional al resultado obtenido. Mejora del modelo (10 puntos): Si el adapter mejora el EX en al menos 0.15 respecto al modelo original, obtiene los 10 puntos. Si la mejora es menor, la nota será proporcional; si no mejora, obtiene 0 puntos.

Penalizaciones:

- Modelo fuera de la lista o API externa: 0 en la evaluación oculta.
- Esquemas de validación presentes en el entrenamiento: −15.
- Salida no determinista: −10.
- Consultas que modifican la base: −10.
- Repositorio sin *commits* de ambos integrantes: 0 en colaboración.

Este proyecto puede ser la base de su Proyecto Final: conserven el adapter, el prompt, el dataset y su evaluador.